

CODE SPORT



In this month's column, we cover how machine learning and natural language processing can be applied to the area of programming.

In this column, we cover computer science interview questions and topics related to natural language processing (NLP) and machine learning (ML). NLP has been applied to various real world applications. For instance, sentiment analysis has been applied to measure customer sentiment from product reviews. Natural language understanding (NLU) has been used to understand human conversations, deciphering the intent of a communication and the entities involved in it. Natural language generation (NLG) has been used to create human-like conversational replies. NLU and NLG are the foundations for designing conversational AI systems like chatbots and virtual agents. NLP has been deployed in various fields such as customer care, health care, finance and transportation.

In spite of the many real world applications of NLP, it is quite surprising that NLP/ML techniques have not been applied to software development extensively. There has been some research in applying these techniques to software development; however, they have not been applied extensively in real world deployment. But with the advent of deep learning, NLP/ML techniques are also getting explored in the area of software development. We will cover this topic over the next few columns. In the area of software development, we will discuss how NLP/ML can be applied to: (a) programming, (b) compilers, and (c) software tools and applications.

Before we get into this topic, it would be good to define and differentiate two terms that are confusing for many lay persons. The two terms 'Learn to Compile' and 'Compile to Learn' are used often in this area. 'Compile to Learn' is all about how specialised/targeted compiler techniques can be used in software development to improve the performance of NLP/ML applications. For instance, TensorFlow has developed a special compiler optimiser called

XLA that targets linear algebra routines, which are often part of deep learning (DL) applications. This is a case of 'Compile to Learn', where compiler techniques are used to improve ML applications. 'Learn to Compile' is completely different from this. Here, the key focus is on how we can use NLP/ML techniques to improve programming and compilers. For example, researchers have used NLP/ML techniques to perform improved loop optimisations in compilers. These techniques are part of 'Learn to Compile', as they are used in compilers to improve the performance of all kinds of software applications. It is important to understand the difference between these two terms. In the next couple of columns, we will focus on the 'Learn to Compile' topic.

We are all familiar with the term 'Big Data'. A related concept is 'Big Code'. In the last decade, a large amount of source code in different programming languages has become publicly available due to open source development as well as the emergence of source code repositories such as GitHub. This has opened up large data sets of source code, which can be used for applying machine learning techniques on them. Discussions related to code and software development in community question answering forums such as Stack Overflow have led to both code and text related to code artefacts becoming available for data mining. 'Big Code' is the idea of applying ML and NLP techniques to huge volumes of publicly available source code and related textual artefacts such as documentation, code related discussions and comments.

The first question we need to answer is the following: NLP/ML techniques have been applied to natural language text – text produced by humans in different languages. However, with 'Big Code' we are interested in applying these techniques to source code in high level languages such as C/C++/Python/